Name : Bhoomi Mangesh Naik
Class : D15C
Roll No. : 34
Practical No. : 8

Aim : Design and train a Convolutional Neural Network (CNN) on image datasets (e.g., MNIST/CIFAR)

Theory :

1. Dataset Source

The dataset used for this experiment is obtained from Kaggle:
MNIST in CSV Dataset
https://www.kaggle.com/datasets/oddrationale/mnist-in-csv

2. Dataset Description

The MNIST dataset contains grayscale images of handwritten digits (0–9) used for image classification tasks.

Dataset Characteristics

- Total Records: 70,000 images (60,000 training + 10,000 testing)
- Image Size: 28 × 28 pixels
- Type: Image dataset (converted into CSV format)
- Learning Type: Supervised learning (Classification)

Features Used

Feature	Description
Pixel Values (0–255)	Intensity values of each pixel in 28×28 image
Label (0–9)	Digit class representing the image

3. Mathematical Formulation

3.1 Convolutional Neural Network (CNN)

A Convolutional Neural Network is a deep learning model specialized for image processing. It uses convolution operations to extract spatial features.

Convolution Operation:

$z=(X*W)+b$

Where:

- XXX = Input image
- WWW = Filter (kernel)
- bbb = Bias
- *** = Convolution operation

Activation Function:

$a=f(z)$

Where:

- f(z) = Activation function (ReLU commonly used in CNNs)

3.2 Pooling Layer

Pooling reduces spatial dimensions and computation.

Max Pooling:

$P=\max(\text{region})$

3.3 Fully Connected Layer

After convolution and pooling, features are flattened and passed to dense layers:

$y=f(Wx+b)$

3.4 Loss Function

For multi-class classification, categorical cross-entropy is used:

$$\boxed{L = -\sum y \log(\hat{y})}$$

Where:

- yyy = actual label
- y^ = predicted probability

3.5 Backpropagation

Weights are updated using gradient descent:

$$\boxed{W = W - \eta \frac{\partial L}{\partial W}}$$

Where:

- η = learning rate
- $\boxed{\frac{\partial L}{\partial W}}$ = gradient of loss

4. Algorithm Limitations

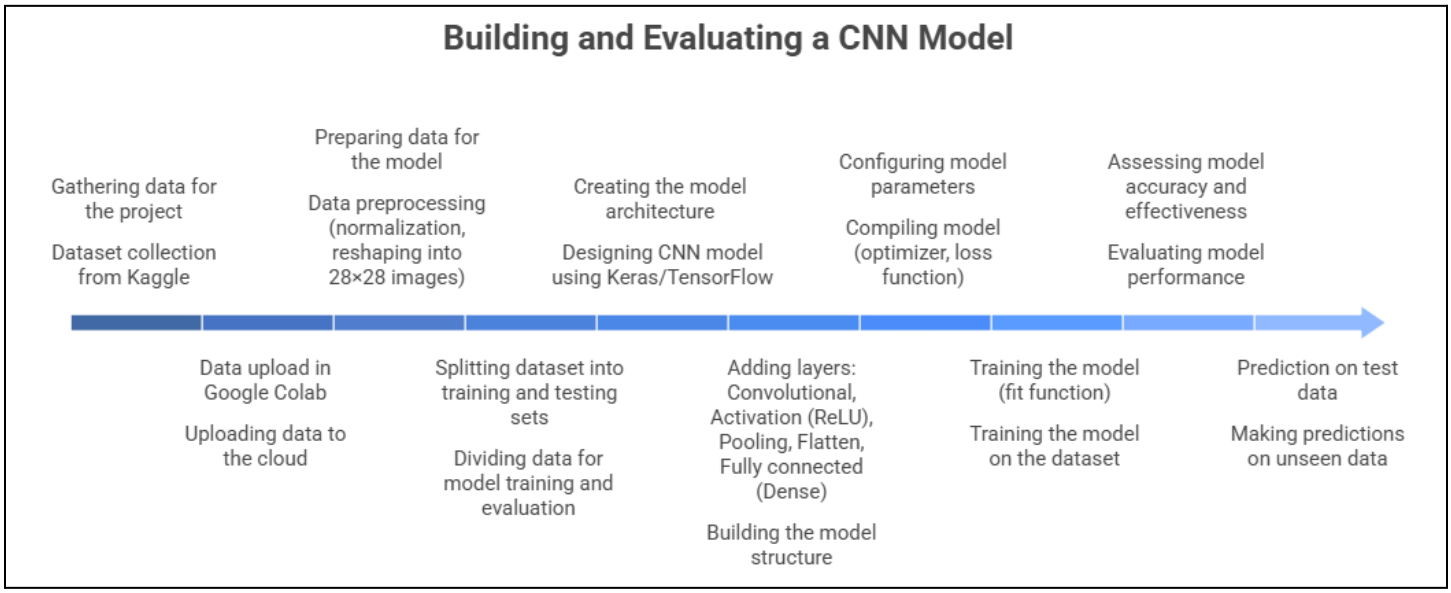
CNN Limitations

- Requires large computational power (GPU preferred)
- Needs large dataset for better generalization
- Training time is high
- Sensitive to hyperparameter selection
- May overfit without regularization (dropout, augmentation)
- Less interpretable compared to traditional models

5. Methodology / Workflow

- Dataset collection from Kaggle
- Data upload in Google Colab
- Data preprocessing (normalization, reshaping into 28×28 images)
- Splitting dataset into training and testing sets
- Designing CNN model using Keras/TensorFlow
- Adding layers:
    - Convolutional layers
    - Activation layers (ReLU)
    - Pooling layers
    - Flatten layer
    - Fully connected (Dense) layers
- Compiling model (optimizer, loss function)
- Training the model (fit function)
- Evaluating model performance
- Prediction on test data

Workflow Diagram:



6. Performance Analysis

- Evaluated using accuracy and loss metrics
- Training vs validation accuracy helps detect overfitting
- Loss curve shows model convergence
- Confusion matrix used for classification performance
- High accuracy indicates effective feature extraction by CNN

7. Hyperparameter Tuning

CNN Hyperparameters

- Number of convolutional layers
- Number of filters (kernels)
- Kernel size
- Pooling size
- Learning rate
- Batch size
- Number of epochs
- Activation functions
- Dropout rate

Impact:

- Improves model accuracy and generalization
- Helps in better feature extraction
- Reduces overfitting and underfitting
- Enhances prediction performance

Code and Output :

```
from google.colab import files
import pandas as pd

# Upload BOTH files: mnist_train.csv and mnist_test.csv
uploaded = files.upload()

train_df = pd.read_csv("mnist_train.csv")
test_df = pd.read_csv("mnist_test.csv")

print("Train Shape:", train_df.shape)
print("Test Shape:", test_df.shape)

# First column = label
y_train = train_df.iloc[:, 0].values
x_train = train_df.iloc[:, 1:].values

y_test = test_df.iloc[:, 0].values
x_test = test_df.iloc[:, 1:].values

print("X_train:", x_train.shape)
print("Y_train:", y_train.shape)
x_train = x_train / 255.0
x_test = x_test / 255.0

x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

print("New Shape:", x_train.shape)
import tensorflow as tf
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import confusion_matrix
import seaborn as sns
```

```
model = keras.Sequential([

    keras.Input(shape=(28,28,1)),

    keras.layers.Conv2D(32, (3,3), activation='relu'),
    keras.layers.MaxPooling2D(2,2),

    keras.layers.Conv2D(64, (3,3), activation='relu'),
    keras.layers.MaxPooling2D(2,2),

    keras.layers.Flatten(),

    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history = model.fit(
    x_train, y_train,
    epochs=10,
    validation_split=0.2
)
test_loss, test_acc = model.evaluate(x_test, y_test)

print("Test Accuracy:", test_acc)
```

```
Train Shape: (60000, 785)
Test Shape: (10000, 785)
```

```
X_train: (60000, 784)
Y_train: (60000,)
```

```
New Shape: (60000, 28, 28, 1)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9914 - loss: 0.0362
Test Accuracy: 0.9914000034332275
```

Google Colab Link for Code and Output : Link for Code and Output

Conclusion :

In this experiment, a Convolutional Neural Network (CNN) was successfully trained on the MNIST dataset for handwritten digit classification. The model effectively learned important features from images using convolution and pooling layers.

The model achieved high accuracy and showed good performance on both training and testing data. This indicates that CNNs are highly efficient and reliable for image classification tasks.